# K-Means Clustering: Theory, Algorithms, and Variations

Compiled from Machine Learning Resources

April 25, 2026

## Introduction to Clustering

Clustering is an **unsupervised learning** task where we aim to discover hidden structure in unlabeled data. Unlike classification, where we have labeled training examples $(x^{(i)}, y^{(i)})$, in clustering we have only features $x^{(i)}$ without corresponding labels.

The goal of clustering is to partition a dataset into groups (clusters) such that:

- Examples within the same cluster are similar to each other,
- Examples in different clusters are dissimilar,
- The number of clusters is either specified or determined by the algorithm.

Clustering has many applications:

- Customer segmentation in marketing,
- Gene sequencing in bioinformatics,
- Document organization and topic discovery,
- Image compression and feature extraction,
- Anomaly detection by identifying outlier clusters.

K-means is one of the most popular and widely-used clustering algorithms due to its simplicity, efficiency, and interpretability.

## 1 Distance Metrics and Similarity

The notion of clustering depends critically on how we measure distance between data points.

### 1.1 Euclidean distance

The most common choice is Euclidean distance:

$$d(x^{(i)}, x^{(j)}) = \left\| x^{(i)} - x^{(j)} \right\|_2 = \sqrt{\sum_{\ell=1}^{n} (x_\ell^{(i)} - x_\ell^{(j)})^2}.$$

Squaring this avoids the square root computation:

$$d^2(x^{(i)}, x^{(j)}) = \sum_{\ell=1}^{n} (x_\ell^{(i)} - x_\ell^{(j)})^2.$$

### 1.2 Other distance metrics

Alternative distance metrics include:

1. **Manhattan distance (L1 norm):** $\sum_\ell |x_\ell^{(i)} - x_\ell^{(j)}|$

2. **Chebyshev distance (L norm):** $\max_\ell |x_\ell^{(i)} - x_\ell^{(j)}|$
3. **Cosine similarity:** $\frac{x^{(i)} \cdot x^{(j)}}{\|x^{(i)}\| \|x^{(j)}\|}$
4. **Mahalanobis distance:** $\sqrt{(x^{(i)} - x^{(j)})^T \Sigma^{-1} (x^{(i)} - x^{(j)})}$

K-means classically uses Euclidean distance, but the algorithm extends naturally to other metrics.

# 2 K-Means Algorithm

## 2.1 Problem formulation

Given a dataset $\{x^{(1)}, \ldots, x^{(m)}\}$ with $x^{(i)} \in \mathbb{R}^n$, we seek to partition the data into $k$ clusters by assigning each point to a cluster center (also called a centroid).

Let:

- $\mu_1, \ldots, \mu_k \in \mathbb{R}^n$ denote the cluster centers,
- $c^{(i)} \in \{1, \ldots, k\}$ denote the cluster assignment of point $x^{(i)}$,
- $r_{ik} \in \{0, 1\}$ denote the indicator that point $i$ belongs to cluster $k$.

The objective is to minimize the within-cluster sum of squares (WCSS):

$$J(c, \mu) = \sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}} \right\|^2 .$$

Equivalently, using the indicator notation:

$$J(r, \mu) = \sum_{i=1}^{m} \sum_{k=1}^{K} r_{ik} \left\| x^{(i)} - \mu_k \right\|^2 .$$

## 2.2 The K-means algorithm

K-means minimizes the objective using coordinate descent, alternating between:

1. **Assignment step:** Assign each point to the nearest cluster center,
2. **Update step:** Recompute cluster centers as the mean of assigned points.

**Algorithm 1** K-Means Clustering

**Input:** Dataset $\{x^{(1)}, \ldots, x^{(m)}\}$, number of clusters $k$, max iterations $T$
Initialize cluster centers $\mu_1, \ldots, \mu_k$ (randomly or using K-means++)
**for** $t = 1$ to $T$ **do**
    **Assignment step:** For each $i \in \{1, \ldots, m\}$,

$$c^{(i)} = \arg \min_{j \in \{1, \ldots, k\}} \left\| x^{(i)} - \mu_j \right\|^2$$

    **Update step:** For each $j \in \{1, \ldots, k\}$,

$$\mu_j = \frac{1}{m_j} \sum_{i : c^{(i)} = j} x^{(i)}, \quad m_j = |\{i : c^{(i)} = j\}|$$

    Check for convergence (cluster assignments unchanged)
**end for**
**Output:** Cluster assignments $c^{(1)}, \ldots, c^{(m)}$ and centers $\mu_1, \ldots, \mu_k$

## 2.3 Assignment step — optimality

For a fixed set of centers $\mu_1, \ldots, \mu_k$, the optimal cluster assignment is:

$$c^{(i)} = \arg\min_j \left\| x^{(i)} - \mu_j \right\|^2,$$

which assigns each point to its nearest centroid. This is a greedy minimization over assignments.

## 2.4 Update step — optimality

For fixed cluster assignments, the optimal centers minimize:

$$J(\mu) = \sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}} \right\|^2.$$

Taking the gradient with respect to $\mu_j$:

$$\frac{\partial J}{\partial \mu_j} = \sum_{i:c^{(i)}=j} 2(x^{(i)} - \mu_j) \cdot (-1) = -2 \sum_{i:c^{(i)}=j} (x^{(i)} - \mu_j).$$

Setting this to zero:

$$\sum_{i:c^{(i)}=j} (x^{(i)} - \mu_j) = 0 \quad \Rightarrow \quad \sum_{i:c^{(i)}=j} x^{(i)} = m_j \mu_j,$$

which gives:

$$\mu_j = \frac{1}{m_j} \sum_{i:c^{(i)}=j} x^{(i)}.$$

This is the **centroid** or mean of all points in cluster $j$.

## 2.5 Convergence guarantee

**Claim:** K-means terminates with a local minimum of the objective $J$.

**Proof sketch:**

The assignment step assigns each point to the nearest center. Any re-assignment that occurs strictly decreases $J$, since we move a point only if its distance to a new center is strictly less.

The update step computes the centroid of each cluster, which minimizes the within-cluster sum of squares for those fixed assignments.

Both steps either decrease $J$ or leave it unchanged. Since $J$ is bounded below (by zero), the algorithm must terminate. Upon termination, no further improvements are possible, so the solution is a local minimum.

**Note:** K-means converges to a local optimum, not necessarily the global optimum. The final result depends on initialization.

## 2.6 Complexity analysis

- **Per iteration:** $O(nkm)$ where $n$ is the feature dimension, $k$ is the number of clusters, and $m$ is the number of examples.
- **Overall:** $O(nkmt)$ where $t$ is the number of iterations (often converges in 10-20 iterations).
- **Space:** $O(m + k)$ for storing data and cluster centers.

This makes K-means practical and scalable for large datasets.

# 3 Initialization Strategies

## 3.1 Random initialization

A naive approach is to randomly select $k$ points from the dataset as initial centers. However, poor initialization can lead to slow convergence or local optima with high cost.

## 3.2 K-means++ initialization

K-means++ is a principled initialization method that often leads to better final clustering and faster convergence.

**Algorithm 2** K-Means++ Initialization

1: Choose the first center $\mu_1$ uniformly at random from the data.
2: **for** $j = 2$ to $k$ **do**
3: For each data point $x^{(i)}$, compute the distance to the nearest center:

$$D(x^{(i)}) = \min_{\ell<j} \left\| x^{(i)} - \mu_\ell \right\|$$

4: Choose the next center $\mu_j$ with probability proportional to $D(x^{(i)})^2$:

$$P(x^{(i)} \text{ is chosen as } \mu_j) = \frac{D(x^{(i)})^2}{\sum_{i'=1}^{m} D(x^{(i')})^2}$$

5: **end for**

**Theoretical guarantee:** K-means++ guarantees that the expected cost is at most $O(\log k)$ times the optimal clustering.

## 3.3 Other initialization methods

- **Forgy method:** Randomly select $k$ examples as initial centers.
- **Random partition:** Randomly assign all points to clusters and compute initial centers.
- **Hierarchical clustering:** Use a dendrogram from hierarchical clustering to select initial centers.

# 4 Determining the Optimal Number of Clusters

A critical question is: how many clusters should we choose?

## 4.1 Elbow method

Plot the within-cluster sum of squares $J$ versus the number of clusters $k$. An "elbow" in the curve suggests the optimal $k$.

$$\text{WCSS}(k) = \sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}}(k) \right\|^2$$

As $k$ increases, WCSS decreases monotonically. The elbow marks where additional clusters provide diminishing returns.

## 4.2 Silhouette coefficient

For each point $x^{(i)}$, define:

- $a^{(i)}$ = average distance to other points in the same cluster,

- $b^{(i)}$ = minimum average distance to points in other clusters.

The silhouette coefficient for point $i$ is:

$$s^{(i)} = \frac{b^{(i)} - a^{(i)}}{\max(a^{(i)}, b^{(i)})}.$$

Values range from -1 to 1:

- $s^{(i)} \approx 1$: Point is well-matched to its cluster,
- $s^{(i)} \approx 0$: Point is near a cluster boundary,
- $s^{(i)} < 0$: Point may be assigned to the wrong cluster.

Average the silhouette coefficients across all points for each $k$. The $k$ with the highest average silhouette score is a good choice.

### 4.3 Gap statistic

The Gap statistic compares the within-cluster sum of squares against a reference null distribution.

Define the gap as:

$$\mathrm{Gap}(k) = \mathbb{E}_{\text{null}}[\log W_k] - \log W_k,$$

where $W_k = \sum_j \sum_{i:c^{(i)}=j} \left\|x^{(i)} - \mu_j\right\|^2$ and the expectation is over random data uniformly distributed in the data range.

Choose $k$ such that $\mathrm{Gap}(k) \geq \mathrm{Gap}(k+1) - s_{k+1}$, where $s_k$ is the standard error.

## 5 K-Means Variants and Extensions

### 5.1 K-medians

Instead of using the mean, K-medians uses the median point (medoid) of each cluster:

$$\mu_j = \mathrm{median}\{x^{(i)} : c^{(i)} = j\}$$

or more generally, the point that minimizes the sum of absolute deviations (L1 distance). K-medians is more robust to outliers than K-means.

### 5.2 K-medoids

K-medoids (or PAM — Partitioning Around Medoids) restricts cluster centers to be actual data points. This ensures that centers are valid examples and avoids the mean, which may not exist in some domains (e.g., categorical data).

The update step becomes:

$$\mu_j = \arg\min_{x \in \text{cluster } j} \sum_{x' \in \text{cluster } j} d(x, x'),$$

where $d$ is any distance metric.

### 5.3 Fuzzy C-means

Instead of hard assignments, Fuzzy C-means (FCM) assigns each point a membership degree $u_{ik} \in [0, 1]$ to each cluster, with $\sum_k u_{ik} = 1$.

The objective becomes:

$$J_m = \sum_{i=1}^{m} \sum_{k=1}^{K} u_{ik}^m \left\|x^{(i)} - \mu_k\right\|^2,$$

where $m > 1$ is a fuzzification parameter.

The update rules are:

$$u_{ik} = \frac{1}{\sum_{j=1}^{K} \left(\frac{d_{ik}}{d_{ij}}\right)^{2/(m-1)}},$$

$$\mu_k = \frac{\sum_{i=1}^{m} u_{ik}^m x^{(i)}}{\sum_{i=1}^{m} u_{ik}^m}.$$

As $m \to 1$, FCM approaches hard K-means. When used in practice, $m = 2$ is common.

### 5.4 Mini-batch K-means

For large datasets, computing the full update step is expensive. Mini-batch K-means processes small random subsets (mini-batches) of the data:

$$\mu_j \leftarrow \frac{n_j}{n_j + s_j}\mu_j + \frac{s_j}{n_j + s_j}\bar{x}_j,$$

where $s_j$ is the number of samples in the mini-batch assigned to cluster $j$, $\bar{x}_j$ is their mean, and $n_j$ is the number of samples ever assigned to cluster $j$.

This is much faster for large $m$ and provides a good initialization for full K-means if needed.

### 5.5 Kernel K-means

K-means can be extended to non-linearly separable data using the kernel trick. The idea is to implicitly map data to a high-dimensional space $\phi(x)$ and run K-means there:

$$J_{\text{kernel}} = \sum_{i=1}^{m} \left\| \phi(x^{(i)}) - \mu_{c^{(i)}}^* \right\|^2.$$

Using the 'kernel trick,' distances can be computed directly in the original space without explicitly computing $\phi$.

## 6 Advantages and Limitations of K-Means

### 6.1 Advantages

- **Simplicity:** Easily understood and implemented.
- **Scalability:** Efficient even for large datasets; complexity is linear in $m$.
- **Interpretability:** Cluster centers are meaningful and interpretable.
- **Fast convergence:** Usually converges in just a few iterations.
- **Flexibility:** Works with any distance metric and feature space.

### 6.2 Limitations

- **Local optima:** May converge to local minima with poor quality.
- **Sensitive to initialization:** Different initializations yield different results.
- **Requires $k$ a priori:** Must specify the number of clusters beforehand; no automatic determination.
- **Assumes spherical clusters:** Works best when clusters are roughly spherical and similarly sized.
- **Sensitive to outliers:** A single outlier can distort a cluster mean.
- **Difficult with mixed sizes:** Clusters of very different sizes are problematic.

# 7 Practical Implementation

## 7.1 Preprocessing and normalization

Before running K-means:

1. **Standardize features:** Subtract mean and divide by standard deviation to ensure all features are on the same scale.
2. **Handle missing values:** Remove or impute them appropriately.
3. **Remove outliers:** Extreme values can significantly affect clustering.

## 7.2 Practical considerations

- Run K-means multiple times with different random initializations and select the clustering with the lowest cost.
- Use K-means++ to reduce the variance due to initialization.
- Consider mini-batch K-means for very large datasets.
- Validate cluster quality using silhouette scores or domain knowledge.

## 7.3 Example: Iris dataset

The Iris dataset contains 150 flowers with 4 features (sepal length, sepal width, petal length, petal width).

Running K-means with $k = 3$:

$$\text{WCSS}(k = 3) \approx 96.5$$
$$\text{Silhouette score} \approx 0.55$$

The algorithm successfully recovers the three Iris species (setosa, versicolor, virginica), though some versicolor and virginica specimens are confused due to their similarity.

# 8 Comparison with Other Clustering Methods

| **Method** | **Scalability** | **Cluster Shape** | **Speed** | **Interpretability** |
|---|---|---|---|---|
| K-means | Excellent | Spherical | Fast | High |
| Hierarchical | Moderate | Any | Slow | High |
| DBSCAN | Good | Any | Moderate | Moderate |
| Gaussian Mixture | Moderate | Elliptical | Moderate | High |
| Spectral | Poor | Any | Slow | Low |

# 9 Conclusion

K-means is a foundational clustering algorithm that balances simplicity, efficiency, and interpretability. While it has limitations (especially regarding local optima and the need to specify $k$), it remains one of the most widely-used algorithms in practice.

Modern extensions like K-means++, fuzzy K-means, and kernel K-means address some limitations. For practitioners, K-means is often the first algorithm to try for exploratory data analysis.

Understanding K-means provides a solid foundation for learning other clustering methods, and its principles appear throughout machine learning, from image quantization to generative models.